Kickoff — Integrating Map Engine V2 into ResQnet

How to start (and steer) the AI agent. This file is for **you**, not for the agent. Do not put it in the agent's rules folder.

Step 0 — Pre-flight (you do this before the agent starts)

1. **Back up.** Commit everything and make a branch or tag (e.g. `before-map-v2`). Not using git? Copy the whole project folder somewhere safe.
2. **Copy the new engine in as-is** to `routing-lab-v2/`, next to the existing `routing-lab/`. Do not merge it into `routing-lab/`. Do not rename either folder. Do not edit anything inside it.
3. **Commit** `routing-lab-v2/` **on its own** so the agent's later changes are easy to review.
4. **Check the app still runs** with the old map, exactly as before.
5. **Put the docs in place:**
   - `rules.md` → the agent's rules location (e.g. `.agents/rules/rules.md`, or use it as `AGENTS.md` in the project root)
   - `memory.md` → project root
6. **Review each change** the agent makes before you accept it.

Then tick "Phase 0" in `memory.md` §2.

Do not tell the agent to replace `routing-lab` "in place". That reads as "delete the old one and move the new one in", which breaks the rules and your working app. The messages below say what to do instead.

Step 1 — Audit only (paste this first)

```
```

```
Read rules.md and memory.md first. Follow rules.md strictly, especially §0 Scope lock.

Context:
- `routing-lab-v2/` is the NEW Map Engine. It is read-only.
- `routing-lab/` is the OLD map/routing. It stays in place and runnable until Phase 5.
- Do NOT edit or delete anything in either folder.

Task: AUDIT ONLY. Do not change any file except memory.md.
1. Explain how the app uses the old map today: which files and pages import from `routing-lab/`, and which routes, APIs and database fields they touch.
2. Explain `routing-lab-v2/`: entry points, services, the packages it needs, env/config it needs, ports.
3. List conflicts between the two: names, ports, packages, global styles, env vars, path aliases.
4. Propose the Phase 1 plan: the files you would create or edit. Flag every file that is outside the rules.md §0 allowed list.
5. Fill in memory.md §4 (open questions), §5 (project facts) and §6 (out-of-scope findings).

Then STOP and wait for my approval. Do not install packages or write code.
```

Read the audit. Answer the open questions. Fix anything that looks wrong **before** approving.

Step 2 — Phase 1: Foundation (after you approve the audit)

```
Approved. Implement Phase 1 (Foundation) exactly as planned in your audit.
- Follow rules.md §0. Ask before touching anything outside the allowed list.
- Keep the old map working. Do not touch `routing-lab/` or edit `routing-lab-v2/`.
- New code imports only from `routing-lab-v2/`, through the adapter.
When done: list every file you changed and why, say which checklist items in memory.md §2 pass and which do not, and update memory.md. Then STOP.
```

Step 3 — Every later phase (reuse this template)

Phase order: 2 Coordinator → 3 Responder → 4 Citizen → 5 Parity & removal.

```
```

```
Phase [N] ([name]) is approved. Implement only Phase [N].
First read memory.md and confirm the previous phase's checklist items pass. State your plan and files to touch, and flag anything outside the §0 allowed list, then wait for my OK.
After my OK: implement, then list every file changed and why, report checklist status, update memory.md, and STOP.
```

Phase 5 removal needs a separate explicit approval from you. The agent must list exactly what it will delete before deleting anything.

If the agent goes off-track

```
□
Stop. List every file you changed outside the rules.md §0 allowed list and why. Do not make any further changes. Wait for my instructions.
```

Then revert those changes yourself (or ask the agent to revert them) and add a line to memory.md §7 (Gotchas) about what happened.